TON_IoT: audit dei duplicati e delle sovrapposizioni

Samuele Marchitelli - 29 settembre 2026

1. Dati, partizioni e provenienza

L'audit riguarda il CSV di rete `train_test_network.csv`: 211.043 record, 44 colonne, 29.902.775 byte. SHA-256: `26ddc513552de36de6428b2e578efaed2b57504c716dfba847cc0109a64e1974`. Il conteggio con il lettore CSV standard concorda con PyArrow. Non risultano valori mancanti secondo il parser impiegato; stringhe come `-` restano valori letterali.

Sono riutilizzati gli indici ricostruiti della prima consegna, identificati dal manifest e verificati tramite hash, senza generare nuovi split. Binary e multiclass hanno ciascuno 168.834 righe training e 42.209 test; le partizioni derivano da stratificazione sul rispettivo target, test 20% e seed 42. Numerosità e classi sono verificate separatamente e non si assume che i due task usino le stesse righe.

La base della pipeline è `852028a`. Il manifest iniziale conserva `8ca44fd` come commit di esecuzione e la prima consegna è `f46365e`; il README ne documenta la relazione. Il nuovo manifest, in `results/full_audit/`, registra `052e748` come HEAD di esecuzione, lo stato del working tree e gli hash dei sorgenti non ancora committati. Il manifest iniziale e i suoi artefatti non sono stati modificati.

2. Metodo

Il confronto A usa le 44 colonne originali, target inclusi. Il confronto B usa le 32 feature sorgente della baseline, escludendo `label`, `type` e le colonne scartate dal trasformatore. Le dieci feature del percorso embedded binary sono esaminate separatamente, sui medesimi indici binary. Ordine e nomi delle colonne sono documentati in `audit_columns.csv`.

L'uguaglianza è esatta sui valori interpretati dal CSV prima del preprocessing; valori mancanti nella stessa colonna sono equiparati. Non si applicano arrotondamenti, imputazione, scaling o ricampionamento. Un gruppo duplicato contiene almeno due righe uguali; le eccedenze sono la somma di `n - 1` per gruppo. Ogni riga di test con almeno una corrispondenza nel training è contata una sola volta. Il denominatore è l'intero test del task, oppure il test della classe nelle tabelle per classe. Non sono stati addestrati modelli né adattati trasformatori.

3. Sovrapposizioni rilevate

Task / confronto	Gruppi condivisi	Righe test esposte	% test
Binary / A	2.831	5.099	12,0804
Binary / B baseline	4.111	6.869	16,2738
Multiclass / A	2.845	5.156	12,2154
Multiclass / B baseline	4.158	6.963	16,4965
Binary / B embedded	6.276	9.071	21,4907

I conteggi completi dei duplicati interni a training e test, delle righe coinvolte e delle eccedenze sono in `audit_summary.csv`; `audit_by_class.csv` contiene il dettaglio per classe. Tutti i CSV citati in questa sezione sono in `results/full_audit/`.

4. Classi e conflitti di etichetta

Nel confronto B baseline multiclass l'esposizione varia sensibilmente: xss 1.594/4.000 (39,85%), ransomware 1.396/4.000 (34,90%), dos 1.380/4.000 (34,50%), contro ddos 5/4.000 (0,125%). Questi valori descrivono la composizione dei dati, non la capacità del classificatore di riconoscere ciascun attacco.

La baseline multiclass presenta 11 gruppi di feature identiche con etichette discordanti, per 43 righe complessive; cinque gruppi sono presenti sia nel training sia nel test. Per le feature embedded binary risultano due gruppi discordanti, per 102 righe, di cui uno condiviso. Non emergono conflitti sul target binary nelle feature baseline. Tali conflitti possono dipendere dall'informazione persa nella selezione delle feature: non dimostrano, da soli, errori di etichettatura.

5. Controlli e limiti

I controlli includono esempi sintetici con conteggi noti, valori mancanti, assenza di sovrapposizioni, distinzione fra righe e coppie corrispondenti, conflitti, precisione numerica e rifiuto di input alterati. Un secondo calcolo con tuple, insiemi e contatori Python verifica ogni campo aggregato e per classe, senza riusare il raggruppamento pandas. Gli esiti eseguibili sono registrati in `validation.json`, che verifica anche la ripetibilità dei CSV e l'integrità della prima consegna. Ambiente: Python 3.11.9 e dipendenze fissate nel repository.

Gli indici sono ricostruiti, non recuperati dagli esperimenti originari; la corrispondenza dei supporti per classe non dimostra l'identità storica delle singole righe. L'audit misura l'esposizione ai duplicati, ma non quantifica l'effetto su accuracy o F1. L'assenza di duplicati esatti non garantisce indipendenza per dispositivo, sessione o tempo. Restano esclusi i fold di tuning, i sottoinsiemi degli esperimenti estesi, SMOTE e le coincidenze dopo preprocessing. La maggiore esposizione embedded riguarda un diverso insieme di feature e non consente di attribuire effetti alla quantizzazione.

6. Proposta per il seguito

Si propone un confronto, da concordare prima di nuovi addestramenti, tra split casuale e split che assegni tutte le righe di ogni gruppo di feature identiche a una sola partizione. Le colonne che definiscono i gruppi devono essere fissate prima della valutazione; vanno documentati gruppi con target discordanti, senza cancellazioni o rietichettature silenziose.

Preprocessing, selezione delle feature e tuning dovranno utilizzare solo training/validation, mantenendo separati i gruppi anche nei fold interni. Il confronto dovrà riportare numerosità, composizione per classe, seed e metriche per classe. Lo split per gruppi potrebbe cambiare proporzioni e difficoltà del test, soprattutto per la classe rara mitm; un'eventuale variazione delle metriche non andrebbe attribuita automaticamente al solo effetto dei duplicati. La verifica di separazione temporale o per dispositivo richiederà evidenze ulteriori.